

LOW-LEVEL DESIGN (LLD) DOCUMENT

1. Introduction

This document describes the internal implementation details of the **RAG-Based Customer Support Assistant**. It focuses on module-level design, data structures, workflow execution, conditional routing, and Human-in-the-Loop (HITL) integration.

2. Module-Level Design

The system is divided into the following modules:

◆ 2.1 Document Processing Module

Responsibility:

- Load PDF document
- Extract text content

Implementation:

- Uses PyPDFLoader

```
loader = PyPDFLoader("RAG.pdf")
docs = loader.load()
```

◆ 2.2 Chunking Module

Responsibility:

- Split document into smaller chunks

Implementation:

- Uses RecursiveCharacterTextSplitter

Configuration:

- Chunk Size: 500
 - Overlap: 50
-

◆ 2.3 Embedding Module

Responsibility:

- Convert text chunks into vector representations

Implementation:

- Uses HuggingFace embedding model (all-MiniLM-L6-v2)

◆ 2.4 Vector Storage Module

Responsibility:

- Store embeddings for fast retrieval

Implementation:

- Uses ChromaDB

```
db = Chroma.from_documents(chunks, embedding)
```

◆ 2.5 Retrieval Module

Responsibility:

- Retrieve relevant chunks based on query

Implementation:

```
retriever = db.as_retriever()  
docs = retriever.invoke(query)
```

◆ 2.6 Query Processing Module

Responsibility:

- Generate response using LLM

Steps:

1. Retrieve relevant chunks
 2. Create context
 3. Generate response
-

◆ 2.7 Graph Execution Module (LangGraph)

Responsibility:

- Manage workflow execution
 - Control node transitions
-

◆ 2.8 HITL Module

Responsibility:

- Handle escalated queries
- Accept human input

3. Data Structures

◆ 3.1 Document Representation

```
{
  "page_content": "text data",
  "metadata": {
    "page": 1
  }
}
```

◆ 3.2 Chunk Format

```
{
  "chunk_id": "unique_id",
  "content": "text chunk",
  "embedding": [0.123, 0.456, ...]
}
```

◆ 3.3 Query-Response Schema

```
{
  "question": "User query",
  "context": "Retrieved text",
  "answer": "Generated response",
  "escalate": false
}
```

◆ 3.4 Graph State Object

```
class GraphState(TypedDict):
    question: str
    context: str
    answer: str
    escalate: bool
```

4. Workflow Design (LangGraph)

◆ Nodes

Node	Function
Process Node	Executes RAG pipeline

Node	Function
HITL Node	Handles human escalation
Output Node	Returns final response

◆ Edges

Process → Output
Process → HITL (conditional)
HITL → Output

◆ State Flow

Question → Context → Answer → Escalation Flag

5. Conditional Routing Logic

◆ Criteria for Answer Generation

- Context is available
 - Response is meaningful
-

◆ Escalation Criteria

Escalation is triggered when:

- No relevant documents found
 - Context length is too small
 - Response contains uncertainty
 - Query is complex
-

◆ Routing Function

```
def route(state):  
    if state["escalate"]:  
        return "hitl"  
    return "output"
```

6. HITL Design

◆ When Escalation Happens

- Low confidence answer
 - Missing context
 - Unknown query
-

◆ Process Flow

User Query → Process Node → Escalate → HITL → Human Response → Output

◆ Integration

```
def hitl_node(state):
    human_response = input("Enter human response: ")
    return {"answer": human_response, "escalate": False}
```

👑 7. API / Interface Design

◆ Input Format

```
{
  "question": "What is refund policy?"
}
```

◆ Output Format

```
{
  "answer": "Refunds are processed within 5 days."
}
```

◆ Interaction Flow

1. User enters query
 2. System processes via LangGraph
 3. Returns response or escalates
-

8. Error Handling

◆ Missing Data

- Return fallback response

- Trigger escalation
-

- ◆ **No Relevant Chunks**

- Set escalate = True
-

- ◆ **LLM Failure**

- Retry or escalate
-

- ◆ **Invalid Input**

- Return error message
-

FINAL SUMMARY

The LLD defines the internal working of the RAG system, including modular design, workflow orchestration, routing logic, and human intervention. It ensures the system is scalable, maintainable, and aligned with real-world customer support use cases.